

Lab0 实验报告

1.

- 1) 之前没有正式的多人开发经历。
- 2) Git 引入暂存区，把修改分为两步：git add 把文件改动放到暂存区，可挑选部分文件与代码片段加入暂存从而灵活筛选提交内容；git commit 把暂存区的改动保存为永久版本记录；可以区分工作区改动和要计入版本库中的改动，不会提交临时改动和垃圾文件；可以选择性提交：只提交文件的一部分；同时可以预览本次保存的变更内容
- 3) Git branch 只列出本地分支，不显示远程仓库的分支；git branch -a(--all)列出所有分支，包括本地分支+远程跟踪分支（远程仓库 origin 上的分支，名称以 remotes/origin/开头）

3.

Git flow 分支控制：

介绍了 gitflow 分支管理策略，指出 git 操作基于分支，多种分支管理策略目的都是保证不同分支各司其职，避免多人协作过程中的代码冲突和版本问题，定义五类关键分支：

master：与线上环境一致，只存放所有发布，每次新 commit 后立即打 tag 记录，为永久分支

develop：开发阶段的操作分支，从 master 分离，新功能开发完成后从 develop 创建 release branch

feature：新功能分支，每次开发新功能从 develop 创建，完成后合并回 develop 并删除

release：测试阶段分支，从 develop 创建后更新版本号，bug fix 完成后合并到 master 和 develop 并删除

hotfix：紧急修复线上 bug 的分支，从 master 或问题 tag 创建，修复后合并到 master 和 develop 并删除

文章强调了一个核心原则：“从哪里来，最后回到哪里去”。此外，版本号格式为 0.0.0 ~ 999.999.999，master 分支上的 tag 不带“v”或“V”

语义化版本：

阐述了语义化版本控制的规范，旨在解决软件依赖管理的“依赖地狱”问题（在依赖关系复杂的系统中，版本控制可能被锁死（升级一个包需要连带修改所有依赖包），或版本混乱（兼容性假设超出合理范围），导致项目维护变得不可靠）

SemVer 的核心规则简洁而严格：版本号格式为 **主版本号.次版本号.修订号** (X.Y.Z)，递增规则如下：

主版本号 (X)：做了不兼容的 API 修改时递增

次版本号 (Y)：做了向下兼容的功能性新增时递增

修订号 (Z)：做了详尽下兼容的问题修正时递增

先行版本号（如 -alpha.1）及版本编译信息可以附加在上述格式之后作为延伸。网页强调，使用这一规范的前提是必须定义好公共 API，因为只有 API 清晰明了，版本号才能准确传达修改的性质。SemVer 的本质是让版本号本身携带信息——使用者仅通过版本号的变化，就能判断升级是否安全、是否需要修改自己的代码。网页还指出，这套规范并非革命性创新，而是对已有惯例的正式化和明确定义：“将上述的想法命名并给予清楚的定义，让你对软件使用者传达意向变得容易”

4.

```
1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6      printf("Hi, ICS!\n");
7  }
```

```
1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6      printf("Hello, ICS!\n");
7  }
```

```
1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6      printf("Goodbye, ICS!\n");
7  }
```

```

1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
        采用当前更改 | 采用传入的更改 | 保留双方更改 | 比较变更
6      <<<<<<< HEAD (当前更改)
7          printf("Goodbye, ICS!\n");
8      =====
9          printf("Hi, ICS!\n");
10     >>>>>>> feature (传入的更改)
11 }

```

```

1  #include <stdio.h>
2
● 3  int main()
4  {
5      // @TODO: print a sentence you want.
6      printf("Goodbye, ICS!\n");
7  }

```

```
李润泽@DESKTOP-K4QDIL4 MINGW64 ~
$ cd newgitlab0

李润泽@DESKTOP-K4QDIL4 MINGW64 ~/newgitlab0 (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

李润泽@DESKTOP-K4QDIL4 MINGW64 ~/newgitlab0 (main)
$ git checkout -b feature
Switched to a new branch 'feature'

李润泽@DESKTOP-K4QDIL4 MINGW64 ~/newgitlab0 (feature)
$ git add main.c

李润泽@DESKTOP-K4QDIL4 MINGW64 ~/newgitlab0 (feature)
$ git commit -m "feat:modify printf on feature branch"
[feature eb472c8] feat:modify printf on feature branch
1 file changed, 1 insertion(+), 1 deletion(-)

李润泽@DESKTOP-K4QDIL4 MINGW64 ~/newgitlab0 (feature)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

李润泽@DESKTOP-K4QDIL4 MINGW64 ~/newgitlab0 (main)
$ git add main.c

李润泽@DESKTOP-K4QDIL4 MINGW64 ~/newgitlab0 (main)
$ git commit -m "main:modify same printf line on main"
[main 321ccc9] main:modify same printf line on main
1 file changed, 1 insertion(+), 1 deletion(-)

李润泽@DESKTOP-K4QDIL4 MINGW64 ~/newgitlab0 (main)
$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.

李润泽@DESKTOP-K4QDIL4 MINGW64 ~/newgitlab0 (main|MERGING)
$ git add main.c

李润泽@DESKTOP-K4QDIL4 MINGW64 ~/newgitlab0 (main|MERGING)
$ git commit -m "merge:resolve conflict between main and feature"
[main c0ed2d6] merge:resolve conflict between main and feature

李润泽@DESKTOP-K4QDIL4 MINGW64 ~/newgitlab0 (main)
$ git push
git@ssh.github.com: Permission denied (publickey).
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.

李润泽@DESKTOP-K4QDIL4 MINGW64 ~/newgitlab0 (main)
$ export GIT_SSH_COMMAND="ssh -i /c/Users/李润泽/id_ed25519"

李润泽@DESKTOP-K4QDIL4 MINGW64 ~/newgitlab0 (main)
$ git push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 22 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 820 bytes | 820.00 KiB/s, done.
```

```
李润泽@DESKTOP-K4QDIL4 MINGW64 ~/newgitlab0 (main)
$ git push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 22 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 820 bytes | 820.00 KiB/s, done.
Total 7 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 2 local objects.
To ssh://ssh.github.com:443/lirunze66/newgitlab0.git
   bc1e16f..c0ed2d6  main -> main
```

```
Code Blame Raw  Copy Download Edit View
1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6      printf("Goodbye, ICS!\n");
7  }
```

实验步骤：

1. 环境准备

在本地 MINGW64 终端配置 SSH 私钥环境变量，指定用于访问 GitHub 的 ed25519 密钥，使用 `export GIT_SSH_COMMAND="ssh -i /c/Users/李润泽/id_ed25519"` 命令，配置 SSH 认证。（环境准备阶段，由于 Windows 系统用户名包含中文，SSH 无法自动检索私钥文件，因此需要通过 `export GIT_SSH_COMMAND` 命令手动指定 ed25519 私钥的完整路径，完成 SSH 访问 GitHub 的认证配置。）

2. 克隆个人仓库

使用带 443 端口的 SSH 协议克隆基于模板仓库创建的个人 GitHub 仓库，执行 `git clone ssh://git@ssh.github.com:443/lirunze66/newgitlab0.git`，将远程仓库下载到本地，得到本地 Git 仓库文件夹。

3. 进入仓库，修改并提交 main.c（基础提交任务）

进入仓库目录 `cd newgitlab0`，打开 main.c 文件，修改文件内 TODO 对应的位置。依次执行：

- `git add main.c`：将修改加入暂存区
- `git commit -m "task1: complete TODO in main.c"`：提交暂存区修改到本地 main 分支
- `git push`：把本地 main 分支提交上传到 GitHub 远程仓库。

4. 创建 feature 分支，在 feature 分支修改 main.c 并提交

执行 `git branch feature` 创建 feature 分支，`git checkout feature` 切换到 feature 分支。修改 main.c 中某一行代码，保存文件。执行 `git add main.c`、`git commit -m "task: modify main.c on feature branch"` 完成本次提交。

5. 切回 main 分支，在 main 分支同一位置修改 main.c 并提交（制造冲突）

执行 `git checkout main` 切回 main 分支。在 main.c 中和 feature 分支修改完全相同的那一行，写入不同内容，保存。执行 `git add main.c`、`git commit -m "task: modify main.c on main branch"` 提交修改。

两个分支修改同一行，后续合并就会触发合并冲突。

6. 执行合并，处理冲突

在 main 分支执行 `git merge feature`，Git 检测到同一行两处不同修改，自动暂停合并，提示冲突。打开 main.c 文件，文件内会出现 <<<<<< HEAD、
=====、>>>>>> feature 标记：

- <<<<<< HEAD 到 =====：main 分支（当前分支）的修改
- =====到>>>>>> feature：feature 分支的修改 手动删除冲突标记，保留需要的代码（可以选择保留其中一份，或者写新内容），保存文件。

7. 冲突解决后提交并推送

冲突修改完成后，执行 `git add main.c` 将解决冲突后的文件加入暂存区；执行 `git commit -m "task: resolve merge conflict between main and feature"` 完成合并提交；执行 `git push`，将合并后的本地 main 分支推送至远程 GitHub 仓库。